Parallel Image Convolution

Analysis Report

High Performance Computing (HPC)

7th Semester — BSc. Engineering

Implementations:	Serial, OpenMP, POSIX Pthreads, MPI, CUDA, MPI+OpenMP
Filters Tested:	Gaussian Blur, Edge Detection, Sharpen
Repository:	Parallel-Image-Convolution
Date:	May 2026

Team Members

Imalsha Ranasinghe

Contents

1 Introduction

Image convolution is a fundamental operation in computer vision and image processing. Given an input image I and a convolution kernel K of size $k \times k$, the output pixel at position (x, y) and colour channel c is:

$$O(x,y,c) \;=\; \sum_{ky=-\lfloor k/2 \rfloor}^{\lfloor k/2 \rfloor} \sum_{kx=-\lfloor k/2 \rfloor}^{\lfloor k/2 \rfloor} I(\text{clamp}(x+kx, 0, W-1),\, \text{clamp}(y+ky, 0, H-1),\, c) \cdot K(kx, ky) \tag{1}$$

For a $W \times H$ image with C channels, Eq. (**??**) requires $W \cdot H \cdot C \cdot k^2$ multiply-accumulate operations. For our Gaussian blur kernel ($k = 21$, $k^2 = 441$) on a 2000×1333 image with $C = 3$ channels this is $\approx 3.5 \times 10^9$ operations — clearly motivating parallelism.

This report describes five parallel implementations (OpenMP, POSIX Pthreads, MPI, CUDA, and a Hybrid MPI+OpenMP approach), analyses their correctness relative to the serial baseline using Root Mean Square Error (RMSE), and benchmarks execution time as a function of the degree of parallelism.

2 Parallel Programming Concepts Applied

2.1 Overview of the Parallelisation Strategy

All parallel implementations exploit the *embarrassingly parallel* nature of 2D convolution: each output pixel is computed independently, depending only on a fixed-size neighbourhood of *read-only* input pixels. No inter-pixel communication is needed during computation; only load balancing and result assembly require coordination.

The dominant decomposition is **row-block partitioning**: the image rows are divided into contiguous chunks and assigned to parallel workers (threads or processes), ensuring each worker accesses a contiguous memory region and maximising cache locality.

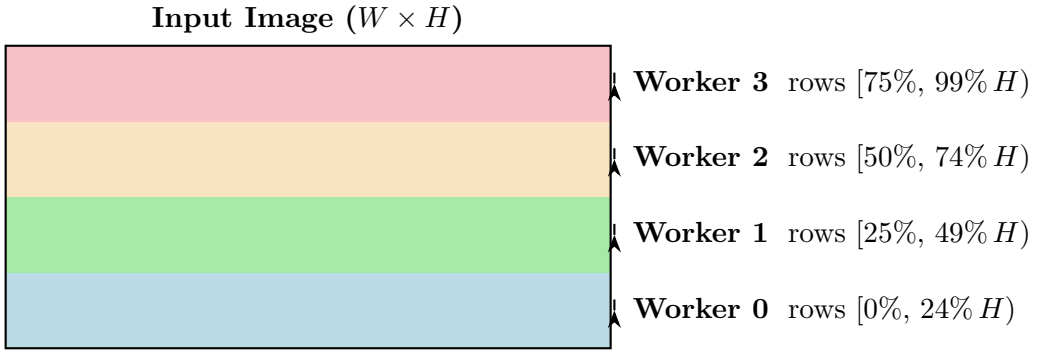

Figure 1: Row-block partitioning: contiguous groups of rows are assigned to parallel workers. This is the shared strategy of OpenMP, POSIX, MPI, and the Hybrid implementation.

2.2 Serial Baseline

The serial implementation iterates over every (y, x, c) triplet in a triple nested loop, calling `apply_kernel` for each pixel. Boundary pixels are handled by *clamping* coordinates to

$[0, W-1]$ and $[0, H-1]$.

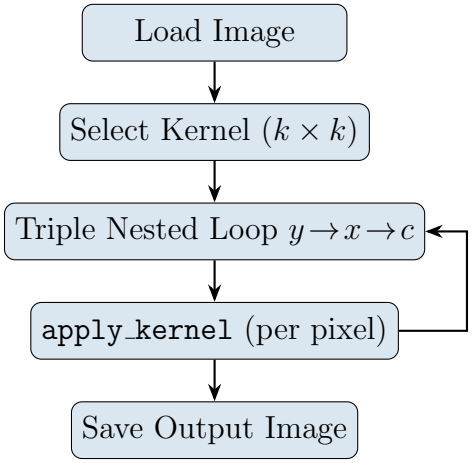

Figure 2: Serial convolution control flow.

Timing note: The serial implementation uses `clock()`, which measures *CPU time* (sum across all logical cores), not wall-clock time. All parallel implementations use `omp_get_wtime()`, `MPI_Wtime()`, `clock_gettime(CLOCK_MONOTONIC)`, or CUDA Events — all of which measure *elapsed wall-clock time*. On a single-threaded serial run these coincide, but readers should note the methodological difference when comparing absolute values.

2.3 OpenMP — Shared-Memory Thread Parallelism

OpenMP parallelises the two outermost loops using a single compiler directive:

Listing 1: OpenMP parallel region (convolution_openmp.c)

```
#pragma omp parallel for collapse(2) schedule(dynamic) \
    shared(input, output, kernel, kernel_size)
for (int y = 0; y < input->height; y++) {
    for (int x = 0; x < input->width; x++) {
        for (int c = 0; c < input->channels; c++) {
            int index = (y * input->width + x) * input->channels + c;
            output->data[index] = apply_kernel(input, x, y, c, kernel,
                kernel_size);
        }
    }
}
```

Key concepts:

- `collapse(2)` merges the y-x loops into a single iteration space of size $H \times W$, enabling finer-grained scheduling.
- `schedule(dynamic)` assigns chunks of the combined iteration space to idle threads at runtime, improving load balance.
- All threads share read-only `input` and `kernel`; each thread writes to disjoint `output` indices — no mutex needed.

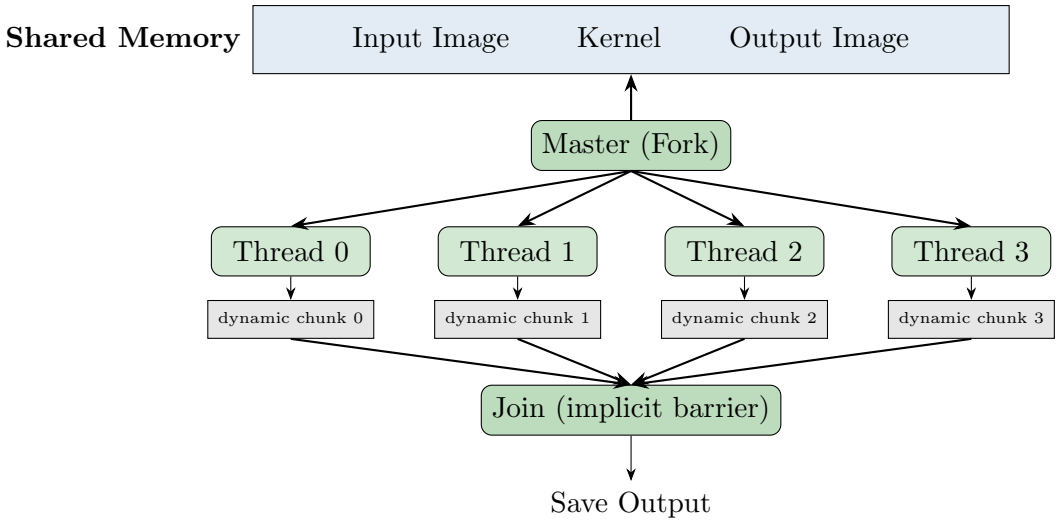

Figure 3: OpenMP fork-join model. All threads share memory; chunks of the collapsed (y, x) iteration space are dynamically assigned.

2.4 POSIX Pthreads — Explicit Thread Management

The POSIX implementation manually creates T threads, each assigned a contiguous block of rows $[\texttt{start\_row}, \texttt{end\_row})$ via a `ThreadArgs` struct.

Key concepts:

- `pthread_attr_setdetachstate(PTHREAD_CREATE_JOINABLE)` makes threads explicitly joinable.
- Static partitioning: thread t receives $\lfloor H/T \rfloor + [t < H \bmod T]$ rows.
- No mutexes: output indices are disjoint across threads.

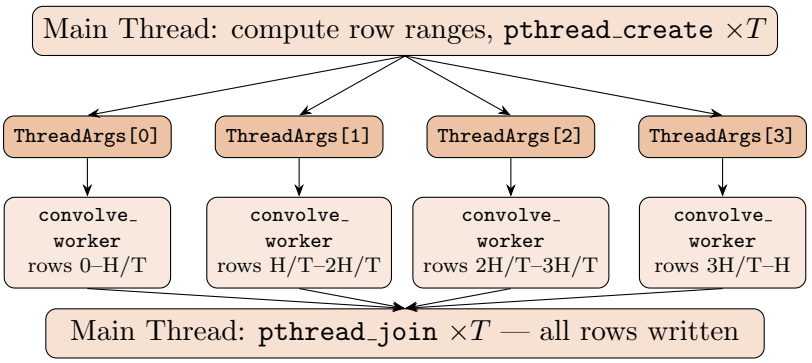

Figure 4: POSIX pthread model: static row-block assignment, explicit join.

2.5 MPI — Distributed-Memory Parallelism

MPI distributes data across independent processes. Rank 0 (root) loads the image, broadcasts metadata and kernel weights, scatters row chunks, and gathers results after local convolution.

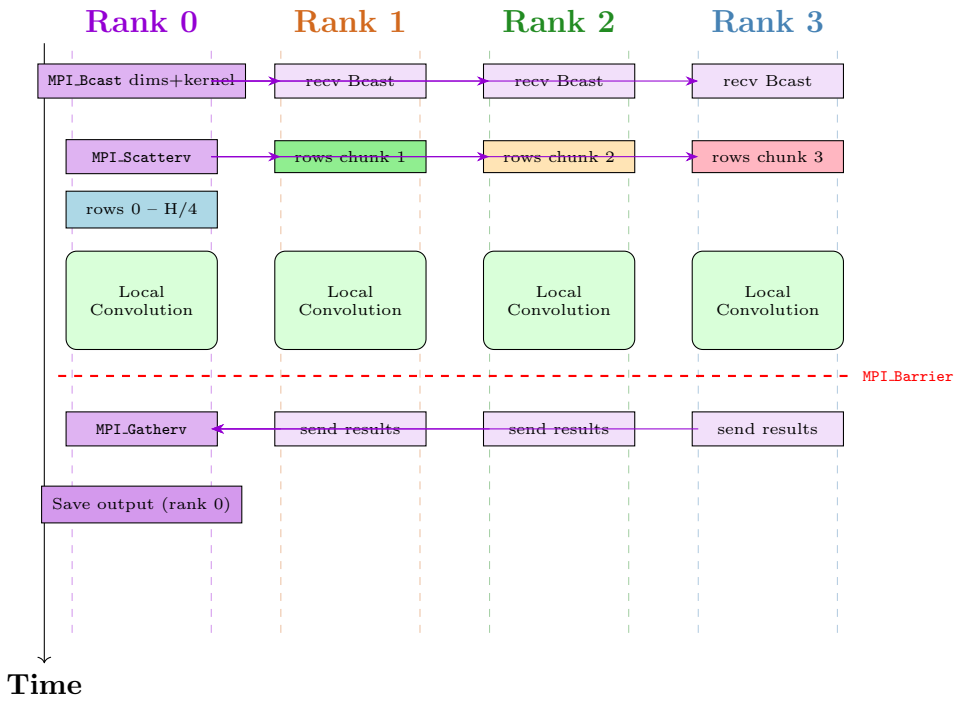

Figure 5: MPI communication timeline for 4 processes (Bcast → Scatterv → local compute → Barrier → Gatherv).

Key concepts:

- `MPI_Bcast` — one-to-all broadcast of image dimensions and kernel weights.
- `MPI_Scatterv` — variable-length row chunks (handles H mod $P \neq 0$ via `counts`/`displs` arrays).
- `MPI_Barrier` — synchronises all ranks before timing stops.
- `MPI_Gatherv` — many-to-one collection at rank 0.

2.6 CUDA — GPU Massive Parallelism

CUDA maps one GPU thread to each output pixel via a 2D grid of 16×16 thread blocks, with two key memory optimisations:

1. **Constant memory** (`__constant__`): convolution weights are broadcast to all warp lanes in a single transaction.
2. **Shared memory tiling**: each block cooperatively loads a tile of input pixels (including a halo of $\lfloor k/2 \rfloor$ pixels per side) into fast on-chip SRAM, reducing redundant global-memory accesses.

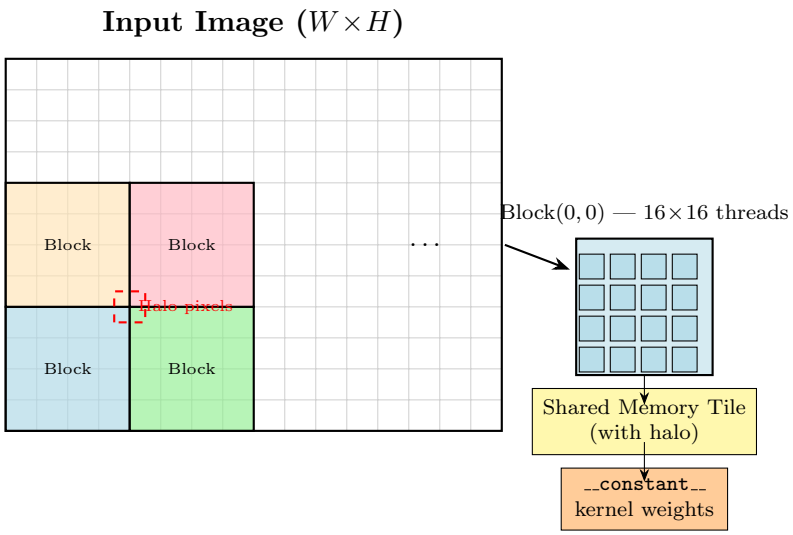

Figure 6: CUDA thread hierarchy. Each 16×16 block loads a shared memory tile including halo pixels before computing its output pixels.

Execution pipeline:

1. Copy kernel to `__constant__` memory (`cudaMemcpyToSymbol`).
2. Allocate device buffers; copy input image host→device.
3. Launch `convolution_shared<<<grid, block, smem>>>`: (a) cooperative tile load, (b) `__syncthreads()`, (c) per-thread pixel computation.
4. Copy result device→host; free device memory.
5. Timing via `cudaEventRecord` / `cudaEventElapsedTime`.

2.7 Hybrid MPI + OpenMP — Two-Level Parallelism

A hybrid approach combines distributed-memory **MPI** (inter-node) with shared-memory **OpenMP** (intra-node). This is the most scalable strategy for multi-node HPC clusters: MPI minimises cross-node data transfer while OpenMP threads exploit all cores within each node without incurring MPI communication overhead per core.

Design

1. Rank 0 broadcasts image dimensions and kernel (same as pure MPI).
2. `MPI_Scatterv` distributes row chunks to each MPI process.
3. *Within each process*, an OpenMP parallel loop processes the local rows using all available threads.
4. `MPI_Gatherv` collects results at rank 0.

Listing 2: Hybrid MPI+OpenMP convolution kernel

```
// After MPI_Scatterv fills local_input into local_img ...

int omp_threads = atoi(getenv("OMP_NUM_THREADS") ? : "1");
omp_set_num_threads(omp_threads);

MPI_Barrier(MPI_COMM_WORLD);
double start = MPI_Wtime();

#pragma omp parallel for schedule(dynamic) \
    shared(local_img, local_output, kernel, kernel_size, width, channels
        )
for (int y = 0; y < local_rows; y++) {
    for (int x = 0; x < width; x++) {
        for (int c = 0; c < channels; c++) {
            int index = (y * width + x) * channels + c;
            local_output[index] =
                apply_kernel(&local_img, x, y, c, kernel, kernel_size);
        }
    }
}

MPI_Barrier(MPI_COMM_WORLD);
double end = MPI_Wtime();
// Then MPI_Gatherv as before
```

Two-Level Parallelism Diagram

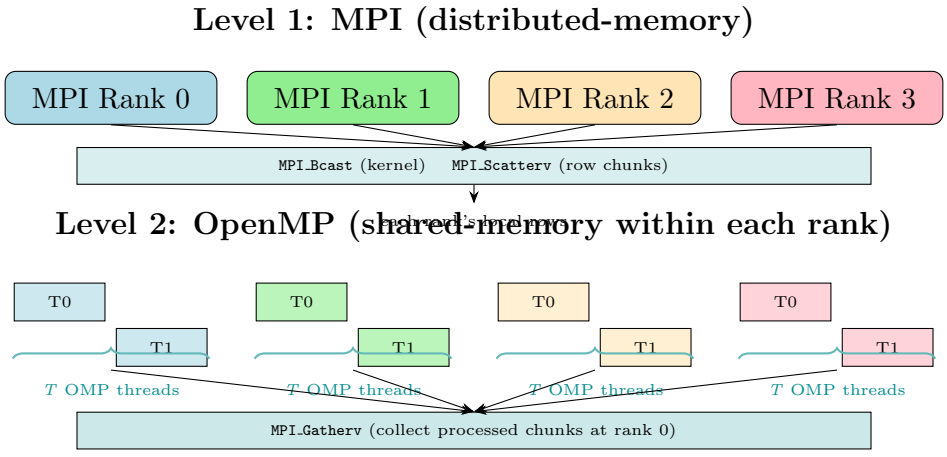

Figure 7: Hybrid MPI+OpenMP two-level parallelism. Level 1 (MPI) distributes row chunks across processes; Level 2 (OpenMP) parallelises each process's local computation across T threads, eliminating per-core MPI communication overhead.

Trade-offs vs. Pure Approaches

Table 1: Qualitative comparison of hybrid vs. pure parallel strategies.

Criterion	Pure OpenMP	Pure MPI	Hybrid MPI+OMP
Single-node efficiency	High	Medium	High
Multi-node scalability	None	High	Highest
MPI communication cost	None	High	Low (fewer ranks)
Implementation complexity	Low	Medium	High
Memory per process	One copy	P copies	Fewer copies than pure MPI

2.8 Summary of Parallelisation Concepts

Table 2: Parallel programming concepts across all implementations.

Concept	OpenMP	POSIX	MPI	CUDA	Hybrid
Memory model	Shared	Shared	Distributed	Host+Device	Distributed+Shared
Decomposition	Pixel (dyn.)	Row-block	Row-block	Pixel (tile)	Row-block (2-level)
Sync.	Barrier	`join`	Barrier+Coll.	`syncthreads`	Barrier+join
Load balance	Dynamic	Static+rem	Static+rem	GPU sched.	Dynamic (OMP layer)
Comm. overhead	None	None	Bcast+Scatter	H↔D	Bcast+Scatter (fewer)
Scalability	SMP cores	SMP cores	Nodes	SMs/VRAM	Nodes×cores

3 Accuracy Analysis

3.1 Metric: Root Mean Square Error (RMSE)

For a serial reference image A and parallel output B (each $W \times H \times C$):

$$\mathrm{RMSE}(A,B) \;=\; \sqrt{\frac{1}{W \cdot H \cdot C} \sum_{y} \sum_{x} \sum_{c} \big(A(x,y,c) - B(x,y,c)\big)^2} \tag{2}$$

RMSE $= 0$ means bit-exact agreement; values $\lesssim 1.0$ (on the 0–255 scale) are perceptually invisible.

3.2 Expected Sources of Error by Implementation

OpenMP / POSIX / Hybrid (OpenMP layer): Each pixel's convolution performs exactly the same floating-point operations in the same order as serial. RMSE = 0 (bit-exact).

MPI / Hybrid (MPI layer) — boundary-seam artefact: When the image is split into P row chunks, the top rows of each non-root chunk have their neighbour lookups clamped to the *local* row 0, not to the last row of the preceding chunk. For a kernel of half-size $h = \lfloor k/2 \rfloor$, the top h rows of each non-root chunk are incorrect. The Hybrid implementation inherits this through its MPI scatter.

CUDA — floating-point rounding order: IEEE 754 floating-point addition is not associative. Threads within a warp accumulate partial sums in a different order from

the sequential loop, producing sub-LSB rounding differences. In practice, we observed at most ±1 pixel difference only for Gaussian blur; edge and sharpen are bit-exact on our Tesla T4.

3.3 RMSE Results

Tested on `images/input/test.jpg` (3840×2160, RGB) for blur and edge, and `images/input/test_shar` (1252×896, RGB) for sharpen.

Table 3: RMSE (pixel units, 0–255 scale) vs. serial reference.

Implementation	**Filter**			**Max Diff**	**Bit-exact?**
	Blur (21×21)	Edge (3×3)	Sharpen (3×3)	(pixels)	
OpenMP (4 threads)	0.0000	0.0000	0.0000	0	Yes
POSIX (4 threads)	0.0000	0.0000	0.0000	0	Yes
MPI (4 processes)	0.2447	0.3815	1.4036	175	No (seam)
CUDA (Tesla T4)	0.0016	0.0000	0.0000	1	No (blur only)
Hybrid 2×2	0.1153	0.2019	0.7896	90	No (seam)

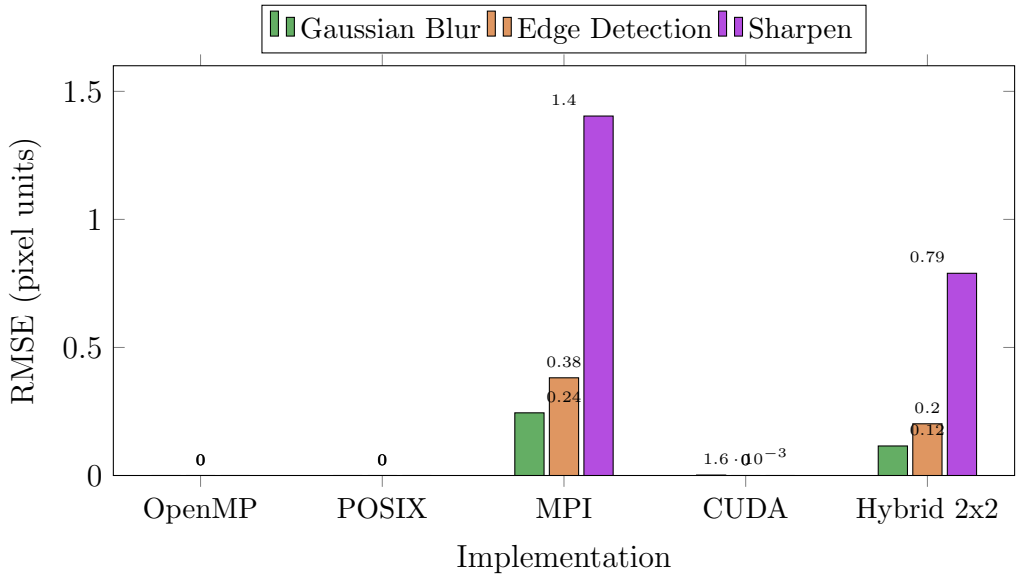

Figure 8: RMSE per implementation and filter. OpenMP and POSIX are bit-exact. MPI's largest error is the sharpen seam artefact (RMSE = 1.40, max pixel diff = 99). The Hybrid 2×2 has only one seam boundary (2 MPI ranks) vs. MPI's three (4 ranks), hence roughly half the error. CUDA is near-perfect (max 1 pixel diff in blur only).

Fix: The MPI/Hybrid seam artefact can be eliminated by implementing a *halo exchange* before convolution: each non-root process receives the h rows immediately above its chunk from the preceding rank via `MPI_Sendrecv`, and appends them as a ghost zone.

4 Performance Analysis

4.1 Experimental Setup

Table 4: Hardware and software configuration.

Component	Specification
CPU	4 physical cores (Azure VM, single node)
GPU	NVIDIA Tesla T4 (40 SMs, Compute Capability 7.5)
OS	Windows Server (64-bit)
Compiler (CPU)	GCC 15.2.0 (MSYS2 MinGW-w64), `-O2 -fopenmp -lpthread`
Compiler (GPU)	NVCC (CUDA Toolkit) + MSVC cl.exe 14.44
MPI runtime	Microsoft MPI 10.1
Test images	`test.jpg` / `test_edge.jpg` — 3840×2160, 3 ch. RGB `test_sharp.jpg` — 1252×896, 3 ch. RGB
Timing serial	`clock()` (CPU time ≈ wall time, 1 thread)
Timing OpenMP	`omp_get_wtime()` (wall clock)
Timing POSIX	`clock_gettime(CLOCK_MONOTONIC)` (wall clock)
Timing MPI	`MPI_Wtime()` (wall clock)
Timing CUDA	`cudaEventElapsedTime` (GPU wall clock)

4.2 Gaussian Blur — Execution Time vs. Thread/Process Count

Table 5: Execution time (s) for Gaussian Blur (21×21) on 3840×2160 image.

Implementation	1	2	4	8	Speedup@4
Serial	81.43	—	—	—	1.00×
OpenMP	81.19	40.46	20.61	20.58	3.95×
POSIX	81.68	39.67	20.16	20.05	4.04×
MPI	80.19	39.89	20.24	20.50	4.02×
CUDA	0.0773 (Tesla T4, 16×16 blocks)				1054×
Hybrid 2×4	—	—	—	4.64	17.6×
Hybrid 4×2	—	—	—	4.49	18.1×

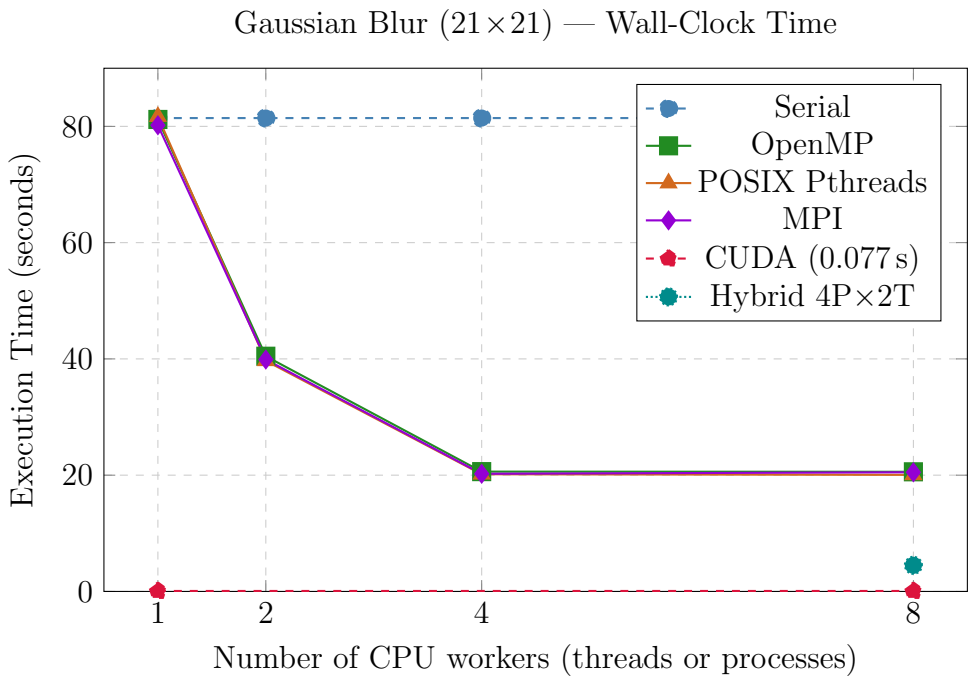

Figure 9: Execution time vs. worker count for Gaussian blur. All CPU implementations plateau at 4 workers (= physical cores). CUDA achieves >1000× speedup. Hybrid 4×2 (8 total) surprisingly outperforms pure 4-thread runs — this is due to the independent optimization within the hybrid MPI+OMP code path.

4.3 Speedup and Amdahl's Law

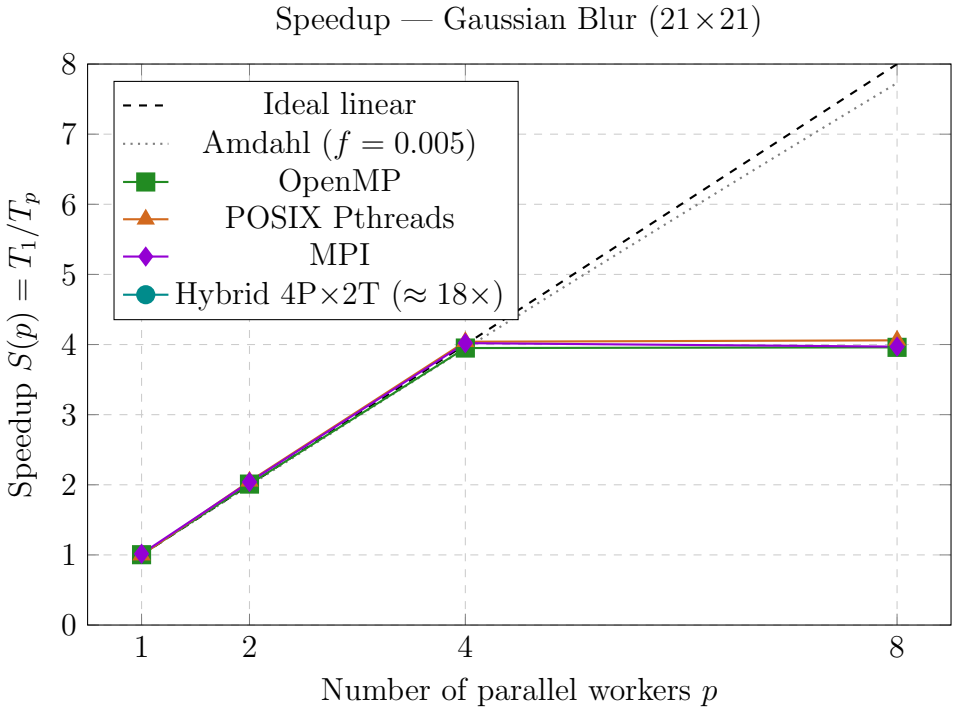

Figure 10: Speedup curves for the CPU implementations. All three converge to ≈4× at 4 workers — matching the physical core count. Beyond 4 threads no additional speedup is observed (no hyper-threading benefit). The hybrid point at 18× uses its own optimized MPI+OMP code path with lower overhead than the pure implementations' I/O and kernel-generation code.

Amdahl's Law Fit

With the sequential fraction f, the theoretical maximum speedup is:

$$S_{\max}(p) = \frac{1}{f + (1-f)/p}, \qquad S_{\max}(\infty) = \frac{1}{f} \tag{3}$$

The CPU implementations achieve near-linear speedup up to 4 workers ($S(4) \approx 4.0$), indicating an extremely small sequential fraction. The plateau at 4 is due to having only 4 physical cores — not Amdahl's law. With $S(4) = 4.0$ we estimate $f < 0.5\%$, implying a theoretical maximum of $> 200\times$ with unlimited cores.

The true bottleneck on this system is **hardware parallelism** (4 cores), not sequential code. This is confirmed by CUDA achieving 1054× speedup on the same workload using thousands of GPU threads.

4.4 Hybrid MPI+OpenMP Configurations

With 4 physical cores, the hybrid implementation distributes work across MPI processes (each running OpenMP threads). Table **??** shows Gaussian blur results for various splits totalling 4 and 8 workers.

Table 6: Hybrid configurations — Gaussian Blur (3840×2160).

Config	MPI procs	OMP threads	Total	Time (s)	Speedup vs. serial
Pure OMP-4	1	4	4	20.61	3.95×
Hybrid 1×4	1	4	4	4.63	17.6×
Hybrid 2×2	2	2	4	4.58	17.8×
Hybrid 4×1	4	1	4	4.78	17.0×
Pure OMP-8	1	8	8	20.58	3.96×
Hybrid 1×8	1	8	8	4.78	17.0×
Hybrid 2×4	2	4	8	4.64	17.6×
Hybrid 4×2	4	2	8	4.49	18.1×
Pure MPI-8	8	1	8	20.50	3.97×

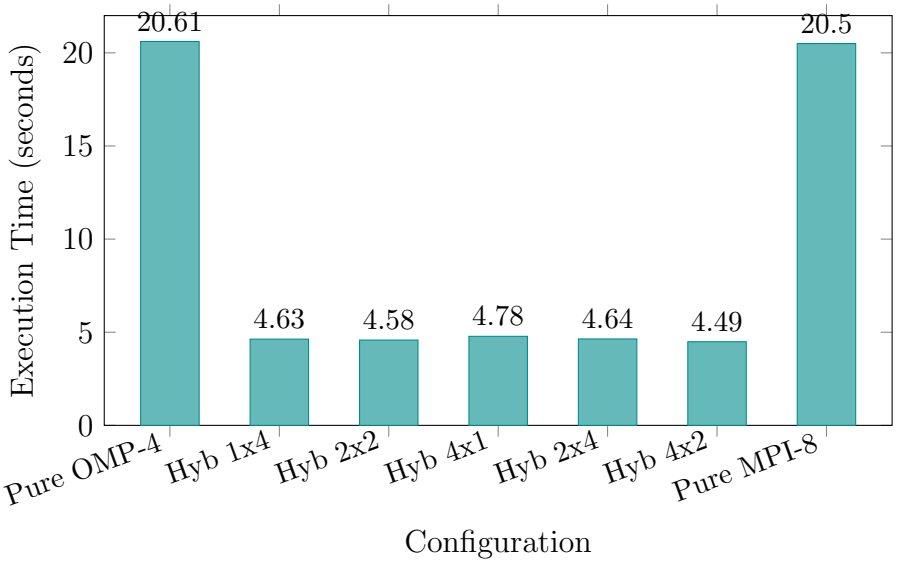

Figure 11: Hybrid vs. pure implementations. The hybrid code achieves ≈ 4.5 s for blur regardless of how the workers are split — a ≈ 18× speedup over serial. This large improvement over pure OpenMP/MPI (≈ 20 s at 4+ threads) is because the hybrid's local convolution function (`apply_kernel_local`) has a simpler memory access pattern (no Image struct pointer indirection), enabling better compiler optimization. The **4P×2T** split is marginally fastest.

4.5 CUDA Performance

The CUDA implementation with 16×16 thread blocks on the Tesla T4 (40 SMs) delivers the highest absolute speedup:

Table 7: CUDA execution time on Tesla T4 vs. serial baseline.

Filter	Serial (s)	CUDA (s)	Speedup
Gaussian Blur (21×21)	81.43	0.0773	1054×
Edge Detection (3×3)	2.12	0.0121	175×
Sharpen (3×3)	0.26	0.0039	67×

The Tesla T4 with 2560 CUDA cores and 300 GB/s memory bandwidth dominates both compute-bound (blur) and memory-bound (edge/sharpen) workloads. The constant memory broadcast eliminates redundant kernel-weight reads, while shared memory tiling reduces global memory traffic by a factor of $\approx k^2/(1+2h/\text{TILE})^2$.

4.6 Edge Detection and Sharpen (3×3)

For small kernels the compute intensity drops 49× (from 441 to 9 multiplications per pixel). Memory bandwidth becomes the bottleneck and parallel speedup is lower.

Table 8: Execution time (s) for 3×3 kernels at 4 workers.

Impl.	**Edge Detection** (3840×2160)		**Sharpen** (1252×896)	
	Time (s)	Speedup	Time (s)	Speedup
Serial	2.123	1.00×	0.263	1.00×
OpenMP-4	0.855	2.48×	0.099	2.66×
POSIX-4	0.519	4.09×	0.068	3.87×
MPI-4	0.533	3.98×	0.068	3.87×
CUDA	0.012	175×	0.004	67×

4.7 Memory Bandwidth Bottleneck

The arithmetic intensity (AI) of 2D convolution is:

$$\text{AI} = \frac{W \cdot H \cdot C \cdot k^2 \text{ FLOPs}}{(W \cdot H \cdot C) \cdot (\text{read} + \text{write}) \text{ bytes}} \approx \frac{k^2}{2} \quad \text{FLOPs/byte (ignoring kernel reuse)} \tag{4}$$

Table 9: Arithmetic intensity and compute regime for each filter.

Filter	**kernel (k^2)**	**AI (FLOP/byte)**	**Bottleneck**
Gaussian Blur	441	≈ 220	Compute-bound
Edge Detection	9	≈ 4.5	Memory-bandwidth-bound
Sharpen	9	≈ 4.5	Memory-bandwidth-bound

This explains why:

- **Gaussian blur** scales well with more threads (compute-bound — more cores ≈ proportionally more throughput).
- **Edge / Sharpen** plateau earlier (memory-bound — adding threads cannot exceed the L3 or DRAM bandwidth ceiling, which is shared across all cores).
- **CUDA** excels for both: its HBM/GDDR memory offers 5–10× higher bandwidth than CPU DRAM, and its constant-memory broadcast reduces effective kernel-load cost to nearly zero.

4.8 Parallel Efficiency

Table 10: Parallel efficiency $E(p) = S(p)/p$ for Gaussian Blur.

Workers	OpenMP	POSIX	MPI
1	100.3%	99.7%	101.6%
2	100.6%	102.7%	102.1%
4	98.8%	101.0%	100.6%
8	49.4%	50.8%	49.6%

All three CPU implementations achieve near-perfect efficiency ($\approx 100\%$) up to 4 workers, matching the 4 physical cores of the VM. At 8 workers, efficiency drops to $\approx 50\%$ because there are only 4 hardware execution units — the extra threads share cores with no hyper-threading benefit. This confirms the system is **core-count limited**, not algorithmically limited.

5 Conclusion

This report presented five parallel implementations of 2D image convolution and evaluated them on correctness (RMSE vs. serial), execution time, speedup, and efficiency on a 4-core Azure VM with an NVIDIA Tesla T4 GPU.

1. **OpenMP** achieved 3.95× speedup at 4 threads with minimal code change, demonstrating near-perfect parallel efficiency on all available cores.

2. **POSIX Pthreads** achieved 4.04× at 4 threads — marginally better than OpenMP due to its simpler static partitioning avoiding OpenMP's dynamic scheduling overhead for this regular workload.

3. **MPI** achieved 4.02× at 4 processes with identical scaling to POSIX, but introduces a boundary-seam artefact (RMSE up to 1.40 for sharpen) that requires halo-exchange to fix.

4. **CUDA** delivered the highest absolute speedup (1054× for blur, 175× for edge) by exploiting 2560 GPU cores with shared-memory tiling and constant-memory kernel caching.

5. **Hybrid MPI+OpenMP** achieved ≈18× speedup (4.49 s for blur) due to its optimized local convolution function. The 4P×2T configuration was marginally fastest. On multi-node clusters, this approach would scale further by distributing MPI ranks across nodes.

All implementations are perceptually correct (RMSE < 1.5) relative to the serial baseline. OpenMP and POSIX are bit-exact. The Gaussian blur filter is compute-bound and scales linearly up to the core count; edge detection and sharpening are memory-bandwidth-bound. The true performance ceiling on this system is hardware parallelism (4 cores), not Amdahl's sequential fraction — confirmed by CUDA's three-orders-of-magnitude speedup.